

# Edit Page

---

We have the "C" and the "R" in CRUD. Now we need to add the "U" for update. We need to create an edit page where we can update the job listing. We already have our methods in place and if you have been following along, you should have the edit button on the details page and it should take you to <http://localhost:8000/jobs/:id/edit>. I know it may be a bit confusing because a lot of other frameworks use the `/job/edit/:id` but we are using the `/jobs/:id/edit`. This is just a convention that Laravel uses.

## Controller Method

We need to fetch the job listing that we want to edit and pass it to the view, which we'll create in a minute.

We can either use the `find` method to find the job listing by the id.

```
public function edit(string $id): View
{
    // Fetch the current job listing
    $job = Job::find($id);
    return view('jobs.edit')->with('job', $job);
}
```

Or, we can just use model binding and pass the job object to the method.

```
public function edit(Job $job): View
{
    return view('jobs.edit')->with('job', $job);
}
```

I will use the latter.

Let's create the edit view. Create a new file in the `resources/views/jobs` directory called `edit.blade.php`. We can copy the create page and make the changes. Here are the changes we need to make.

### Heading

Change the heading to "Edit Job Listing":

```
<h2 class="text-4xl text-center font-bold mb-4">Edit Job Listing</h2>
```

### Action & @method Directive

We need to change the form action to point to the update route. We can use the `route` helper function to generate the URL. We need to pass in the route name and the job listing id. Here is the updated form tag:

```
<form
  action="{{ route('jobs.update', $job->id) }}"
  method="POST"
  enctype="multipart/form-data"
></form>
```

We also need to add the `@method` directive to tell Laravel that we are using the `PUT` method. Regular HTML forms can only use the `GET` and `POST` methods. We can use the `@method` directive to tell Laravel that we are using the `PUT` method. Here is the updated form tag:

```
<form
  method="POST"
  action="{{ route('jobs.update', $job->id) }}"
  enctype="multipart/form-data"
>
  @csrf
  <!-- Add this line -->
  @method('PUT')
</form>
```

## Values

For the values, we want the current job listing values. We can just pass the value into each input component. Here is the final code for the edit form:

```
<x-layout>
  <div class="bg-white mx-auto p-8 rounded-lg shadow-md w-full md:max-w-3xl">
    <h2 class="text-4xl text-center font-bold mb-4">Edit Job Listing</h2>

    <!-- Form Start -->
    <form
      method="POST"
      action="{{ route('jobs.update', $job->id) }}"
      enctype="multipart/form-data"
    >
      @csrf @method('PUT')

      <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">
        Job Info
      </h2>

      <!-- Job Title -->
      <x-inputs.text
        id="title"
        name="title"
        label="Job Title"
        placeholder="Software Engineer"
```

```

      :value="old('title', $job->title)"
    />

    <x-inputs.textarea
      id="description"
      name="description"
      label="Job Description"
      placeholder="We are seeking a skilled and motivated Software Developer..."
      :value="old('description', $job->description)"
    />

    <x-inputs.text
      id="salary"
      name="salary"
      label="Annual Salary"
      type="number"
      placeholder="90000"
      :value="old('salary', $job->salary)"
    />

    <x-inputs.textarea
      id="requirements"
      name="requirements"
      label="Requirements"
      placeholder="Bachelor's degree in Computer Science"
      :value="old('requirements', $job->requirements)"
    />

    <x-inputs.textarea
      id="benefits"
      name="benefits"
      label="Benefits"
      placeholder="Health insurance, 401k, paid time off"
      :value="old('benefits', $job->benefits)"
    />

    <x-inputs.text
      id="tags"
      name="tags"
      label="Tags (comma-separated)"
      placeholder="development,coding,java,python"
      :value="old('tags', $job->tags)"
    />

    <x-inputs.select
      id="job_type"
      name="job_type"
      label="Job Type"
      :options="['Full-Time' => 'Full-Time', 'Part-Time' => 'Part-Time',
'Contract' => 'Contract', 'Temporary' => 'Temporary', 'Internship' =>
'Internship', 'Volunteer' => 'Volunteer', 'On-Call' => 'On-Call']"
      :value="old('job_type', $job->job_type)"
    />

```

```
<x-inputs.select
  id="remote"
  name="remote"
  label="Remote"
  :options="[0 => 'No', 1 => 'Yes']"
  :value="old('remote', $job->remote)"
/>

<h2 class="text-2xl font-bold mb-6 text-center text-gray-500">
  Company Info
</h2>

<x-inputs.text
  id="address"
  name="address"
  label="Address"
  placeholder="123 Main St"
  :value="old('address', $job->address)"
/>

<x-inputs.text
  id="city"
  name="city"
  label="City"
  placeholder="Albany"
  :value="old('city', $job->city)"
/>

<x-inputs.text
  id="state"
  name="state"
  label="State"
  placeholder="NY"
  :value="old('state', $job->state)"
/>

<x-inputs.text
  id="zipcode"
  name="zipcode"
  label="ZIP Code"
  placeholder="12201"
  :value="old('zipcode', $job->zipcode)"
/>

<x-inputs.text
  id="company_name"
  name="company_name"
  label="Company Name"
  placeholder="Company name"
  :value="old('company_name', $job->company_name)"
/>

<x-inputs.textarea
  id="company_description"
```

```

        name="company_description"
        label="Company Description"
        placeholder="Company Description"
        :value="old('company_description', $job->company_description)"
    />

    <x-inputs.text
        id="company_website"
        name="company_website"
        label="Company Website"
        type="url"
        placeholder="Enter website"
        :value="old('company_website', $job->company_website)"
    />

    <x-inputs.text
        id="contact_phone"
        name="contact_phone"
        label="Contact Phone"
        placeholder="Enter phone"
        :value="old('contact_phone', $job->contact_phone)"
    />

    <x-inputs.text
        id="contact_email"
        name="contact_email"
        label="Contact Email"
        type="email"
        placeholder="Email where you want to receive applications"
        :value="old('contact_email', $job->contact_email)"
    />

    <x-inputs.file
        id="company_logo"
        name="company_logo"
        label="Company Logo"
    />

    <!-- Submit Button -->
    <button
        type="submit"
        class="w-full bg-green-500 hover:bg-green-600 text-white px-4 py-2 my-3
rounded focus:outline-none"
    >
        Save
    </button>
</form>
</div>
</x-layout>

```

## Update Method

Now we handle the actual update. This will be very similar to the `store` method. There are some changes though. First off, let's use model binding. Instead of passing in the `id` as a parameter, we can just pass in the model itself along with the request object.

```
public function update(Request $request, Job $job){}
```

We also want to delete the image before we update it if a new logo was submitted. To do this, we need to use the `Storage` facade. We need to import the `Storage` facade at the top of the file:

```
use Illuminate\Support\Facades\Storage;
```

```
// Check if a file was uploaded
if ($request->hasFile('company_logo')) {
    // Delete the old company logo from storage
    if ($job->company_logo) {
        Storage::delete('public/logos/' . basename($job->company_logo));
    }
    // Store the file and get the path
    $path = $request->file('company_logo')->store('logos', 'public');

    // Add the path to the validated data array
    $validatedData['company_logo'] = $path;
}
```

Finally, we want to call the `update` method on the model. We can use the `$job` variable:

```
// Update with the validated data
$job->update($validatedData);
```

Here is the entire method:

```
public function update(Request $request, Job $job): RedirectResponse
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'required|string',
        'salary' => 'required|integer',
        'tags' => 'nullable|string',
        'job_type' => 'required|string',
        'remote' => 'required|boolean',
        'requirements' => 'nullable|string',
        'benefits' => 'nullable|string',
    ]);

    $job->update($validatedData);

    return redirect($job->url());
}
```

```

        'address' => 'nullable|string',
        'city' => 'required|string',
        'state' => 'required|string',
        'zipcode' => 'required|string',
        'contact_email' => 'required|email',
        'contact_phone' => 'nullable|string',
        'company_name' => 'required|string|max:255',
        'company_description' => 'nullable|string',
        'company_logo' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
        'company_website' => 'nullable|url',
    ]);

    // Check if a file was uploaded
    if ($request->hasFile('company_logo')) {
        // Delete the old company logo from storage
        if ($job->company_logo) {
            Storage::delete('public/logos/' . basename($job->company_logo));
        }
        // Store the file and get the path
        $path = $request->file('company_logo')->store('logos', 'public');

        // Add the path to the validated data array
        $validatedData['company_logo'] = $path;
    }

    // Update with the validated data
    $job->update($validatedData);

    return redirect()->route('jobs.index')->with('success', 'Job listing updated successfully!');
}

```

I also removed the following line:

```
$validatedData['user_id'] = 1;
```

We will keep the user ID as what is stored in the database.

Now try and edit a job listing. Try without changing the logo first and then try changing the logo. You should see the logo change on the details page and the old one should get deleted. You should also see the success message.

In the next lesson, we will add the delete functionality.